

SECURITY ASSESSMENT



SHIVAM KUMAR
CBS-0249

Project Scenario

Overview

As the lead security engineer for CryptoV4ult, a prominent international cryptocurrency platform, you're tasked with ensuring the security and integrity of our newly established infrastructure. With over 1 million users relying on our services, it's imperative that we maintain the highest standards of security to protect their digital assets.

Your role involves a comprehensive review of the security landscape for our new application technology stack, identifying potential vulnerabilities, and running scans to assess any existing threats. Your scope encompasses various entities within our architecture, including the application itself, containerized services, and the external-facing API.

Ultimately, your objective is to develop a robust remediation plan that not only addresses current vulnerabilities but also strengthens our overall security posture, safeguarding both user data and the platform's reputation. This critical mission presents an exciting opportunity to leverage your skills and expertise in cybersecurity to fortify our infrastructure and uphold our commitment to providing a secure and reliable platform for our users. Let's embark on this journey together to ensure CryptoV4ult remains a trusted leader in the cryptocurrency industry!

Section I:

Integrating SDLC

Transitioning to Secure SDLC

As the lead security engineer at CryptoV4ult, you are tasked with ensuring the new infrastructure is developed securely. Your responsibility is to reorganize the existing development tasks to fit into a Secure Software Development Lifecycle (SDLC) framework, ensuring that each stage of the lifecycle incorporates necessary security tasks to protect user data and maintain the integrity of the cryptocurrency platform.

Transitioning to Secure SDLC

Place every task into a Secure SDLC category in the next few slides. Add at least one additional task to each phase that helps enhance security.

1. Conduct user interviews to gather functional requirements.
2. Write a requirements document for task management features.
3. Create a high-level architecture diagram for the application.
4. Design the database schema for tasks.
5. Code the user interface using HTML and CSS.
6. Implement interactive elements using JavaScript.
7. Set up a Flask application to handle API requests.
8. Implement CRUD operations for tasks.
9. Write and execute functional test cases.
10. Conduct browser compatibility testing.
11. Deploy the application to Heroku.
12. Perform smoke testing on the deployed application.
13. Monitor application logs and fix reported issues.
14. Gather user feedback for future feature additions.

Transitioning to Secure SDLC

Requirements Analysis

- Conduct user interviews to gather functional requirements.
- Write a requirements document for task management features.
- Define security requirements (e.g., authentication rules, data compliance needs).

Design

- Create a high-level architecture diagram for the application.
- Design the database schema for tasks.
- Perform Threat Modeling to identify potential attack vectors in the architecture.

Transitioning to Secure SDLC

Development

- Code the user interface using HTML and CSS.
- Implement interactive elements using JavaScript.
- Set up a Flask application to handle API requests.
- Implement CRUD operations for tasks.
- Conduct Static Application Security Testing (SAST) to find code vulnerabilities early.

Testing

- Write and execute functional test cases.
- Conduct browser compatibility testing.
- Perform Dynamic Application Security Testing (DAST) on the running application.

Transitioning to Secure SDLC

Deployment

- Deploy the application to Heroku.
- Perform smoke testing on the deployed application.
- Configure secure server environments (harden OS, disable unused ports) before go-live.

Maintenance

- Monitor application logs and fix reported issues.
- Gather user feedback for future feature additions.
- Conduct periodic penetration testing and vulnerability scanning.

Advocating for Secure SDLC

As the lead security engineer at CryptoV4ult, you're spearheading the shift towards a more secure and agile development process. To get everyone on board, create a succinct list highlighting five essential advantages of transitioning to the Secure Software Development Lifecycle (SDLC) from our current Waterfall methodology. **For each advantage, include a brief explanation** that underscores its importance, particularly focusing on how it benefits the dynamic and security-centric nature of our cryptocurrency platform.

Advocating for Secure SDLC

1. Early Vulnerability Detection

Integrating security early ("shifting left") allows us to find and fix flaws during design/coding rather than post-deployment, which is crucial for protecting user crypto assets.

2. Cost Reduction

Fixing security bugs during the development phase is significantly cheaper (up to 30x) than remediating them after the platform is live and user funds are at risk.

3. Regulatory Compliance

Built-in security controls ensure we consistently meet strict financial regulations (like GDPR or PCI-DSS) required for a cryptocurrency platform, avoiding fines.

4. Enhanced Trust & Reputation

A secure-by-design platform builds user confidence. In the crypto industry, a single breach can destroy reputation; Secure SDLC minimizes this risk.

5. Streamlined Incident Response

[Continuous monitoring and documentation (part of SDLC maintenance) ensure that if a threat emerges, our team can respond and patch it faster than with ad-hoc security.]

Section 2:

Vulnerabilities and Remediation

Vulnerabilities and remediation

As CryptoV4ult enhances its infrastructure to support new features for its extensive user base, ensuring the security of user authentication mechanisms is paramount. The **login system** is critical to the platform's security, acting as the first line of defense against unauthorized access. Your task is to scrutinize a login system, **identify 3 potential vulnerabilities** they usually have, and propose effective remediation strategies.

- *Concentrate on **login systems in general***
- *The vulnerability can relate to any aspect of a login system, including user identification, authentication mechanisms, and session management*
- *Any common login system vulnerability is acceptable*
- ***For each identified potential vulnerability, you need to:***
 - ***Describe the vulnerability***
 - ***Explain the risk***
 - ***Provide remediation strategy***

Vulnerabilities and remediation

1.Brute Force / Credential Stuffing

Description

Attackers use automated tools to guess passwords or try stolen credentials.

Risk

Unauthorized access to user wallets, leading to theft of funds.

Remediation

Implement Account Lockout (e.g., after 5 failed attempts) and Rate Limiting (CAPTCHA) to stop automated attacks. Enforce MFA.

Vulnerabilities and remediation

2. Broken Authentication (Session Hijacking)

Description

Weak session IDs are exposed in URLs or not invalidated after logout.

Risk

Attackers can takeover a valid user session and perform transactions as that user.

Remediation

Use secure, random session cookies with `HttpOnly` and `Secure` flags. Ensure immediate session invalidation upon logout.

Vulnerabilities and remediation

3. SQL Injection (Login Form)

Description

Malicious SQL code is entered into login fields (e.g., 'OR '1'='1').

Risk

Attackers can bypass authentication entirely without a password to access admin accounts.

Remediation

Use Parameterized Queries (Prepared Statements) for all database interactions. Validate and sanitize all user inputs.

Create a threat Matrix

Dissect and categorize the 3 vulnerabilities that you have identified for the login system. Understanding these vulnerabilities from a strategic viewpoint will enable the company to allocate resources efficiently, prioritize remediation efforts, and maintain CryptoV4ult's reputation as a secure and reliable platform.

- *For each identified vulnerability, critically assess its potential to disrupt CryptoV4ult's operational functionality, erode customer trust, and impact financial stability. **Assign an impact level of 'Low', 'Medium', or 'High'** based on the evaluated potential consequences.*
- *Analyze the complexity and feasibility of exploiting each identified vulnerability. Consider the sophistication required for exploitation and the accessibility of the vulnerability to potential attackers. **Rate the likelihood of exploitation as 'Low', 'Medium', or 'High'**.*
- *Utilize the provided risk matrix framework to **map out the vulnerabilities** according to your assessments of their impact and exploit likelihood.*

Threat Matrix

Pathway (Vulnerability) Impact Level Likelihood Level

Impact	Low	Medium	High
Likelihood			
High			BRUTEFORCE
Medium			Broken Authentication
Low			SQL Injection

Section 3:

Container Security

Container Security

It is time to delve into the container services underpinning CryptoV4ult's application infrastructure by scanning for potential vulnerabilities. Scan one of the container services running in the application (located at `vulnerables/cve-2014-6271`) and identify potential vulnerabilities. Then, you will build a remediation plan to resolve some of the container vulnerabilities.

- *Using **Trivy**, run a **scan** against the container located at **vulnerables/cve-2014-6271**. You can run this scan from the Kali VM in the lab where Trivy is located or from your own computer*
- *Create a **screenshot** of the **Trivy scan results** (it does not have to show all the results) and place it on the next slide*
- ***Fill out the Report to Fix Container Issues** with at least 7 items*

Trivy scan screenshot

Place a screenshot from the Trivy scan results on this slide. Make sure your hostname is clearly visible.

(shivam@KaliLinux-2)-[~]

\$ trivy image vulnerables/cve-2014-6271

2026-01-07T00:19:45:05:30

INFO

[vuln] Vulnerability scanning is enabled

2026-01-07T00:19:45:05:30

INFO

[secret] Secret scanning is enabled

2026-01-07T00:19:45:05:30

INFO

[secret] If your scanning is slow, please try '--scanners vuln' to disable secret scanning

2026-01-07T00:19:45:05:30

INFO

[secret] Please see <https://trivy.dev/dev/docs/scanner/secret#recommendation> for faster secret detection

2026-01-07T00:19:54:05:30

INFO

Detected OS family="debian" version="7.11"

2026-01-07T00:19:54:05:30

INFO

[debian] Detecting vulnerabilities ... os_version="7" pkg_num=118

2026-01-07T00:19:54:05:30

INFO

Number of language-specific files num=0

2026-01-07T00:19:54:05:30

WARN

Using severities from other vendors for some vulnerabilities. Read <https://trivy.dev/dev/docs/scanner/vulnerability>

2026-01-07T00:19:54:05:30

WARN

This OS version is no longer supported by the distribution family="debian" version="7.11"

2026-01-07T00:19:54:05:30

WARN

The vulnerability detection may be insufficient because security updates are not provided

Report Summary

Target	Type	Vulnerabilities	Secrets
vulnerables/cve-2014-6271 (debian 7.11)	debian	254	-
/etc/ssl/private/ssl-cert-snakeoil.key	text	-	1

Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)

vulnerables/cve-2014-6271 (debian 7.11)

Total: 254 (UNKNOWN: 6, LOW: 51, MEDIUM: 72, HIGH: 78, CRITICAL: 47)

Library	Vulnerability	Severity	Status	Installed Version	Fixed Version	Title
apache2	CVE-2018-1312	CRITICAL	fixed	2.2.22-13+deb7u12	2.2.22-13+deb7u13	httpd: Weak Digest auth nonce genera
tion in mod_auth_digest						https://avd.aquasec.com/nvd/cve-2018-1312

						https://avd.aquasec.com/nvd/cve-2017-15710
g the	CVE-2018-1301	MEDIUM				httpd: Out of bounds access after fa
						HTTP request ...
						https://avd.aquasec.com/nvd/cve-2018-1301
orker	CVE-2018-1312	CRITICAL				httpd: Weak Digest auth nonce genera
h_digest						https://avd.aquasec.com/nvd/cve-2018-1312
using too	CVE-2017-15710	HIGH				httpd: Out of bounds write in mod_au
						small Accept-Language ...
						https://avd.aquasec.com/nvd/cve-2017-15710
	CVE-2018-1301	MEDIUM				httpd: Out of bounds access after fa

Report to Fix Container Issues

Fill out the report with 7 items. Make sure to write the **Issues in the correct form of (Application Name: CVE number)**.

Vulnerability Name	Unpatched Software Version	Patched Software Version
apache2: CVE-2018-1312	2.2.22-13+deb7u12	2.2.22-13+deb7u13
apache2: CVE-2017-15710	2.2.22-13+deb7u12	(Check fixed version online or use 'None')
apache2: CVE-2018-1301	2.2.22-13+deb7u12	2.2.22-13+deb7u13
bash: CVE-2014-6271	4.3-7	4.3-11+
bash: CVE-2014-7169	4.3-7	4.3-11+
coreutils: CVE-2016-2781	8.23-4	8.26-3
login: CVE-2013-4235	4.2-3	4.5-1

Section 4:

API Security

API Security

Management has partnered with an external sales vendor and asked for a generic API to be developed that tracks user's data. Based on the data ingested they will create targeted sales advertisements to the customer base, this means a lot of confidential info about the users will be shared to 3rd party vendors.

You need to **identify 3 common API vulnerabilities** and propose effective remediation strategies. Keep in mind this code does not exist; this is the initial stages of development, and you are providing guidance to the engineering team. Feel free to make any assumptions about API features, implementations, and what private data might be shared.

- ***For each identified common API vulnerability:***
 - ***Describe the vulnerability***
 - ***Explain the risk***
 - ***Provide remediation strategy***

Vulnerabilities and remediation

1. Broken Object Level Authorization (BOLA)

Description

API allows accessing data of other users by simply changing an ID in the URL (e.g., `/user/123` to `/user/124`).

Risk

Exposure of confidential user profiles/history to 3rd parties.

Remediation

Implement authorization checks on every object access. Ensure the logged-in user has permission to view the specific ID requested.

Vulnerabilities and remediation

2. Excessive Data Exposure

Description

API returns full data objects (e.g., including PII/SSN) to the client, relying on the frontend to hide it.

Risk

Sales vendors could scrape "hidden" confidential info (PII) meant only for internal use.

Remediation

Filter data on the server-side. Define strict schemas for API responses that return only the specific fields required by the vendor.

Vulnerabilities and remediation

3. Lack of Rate Limiting (DoS)

Description

API accepts unlimited requests from a single source.

Risk

Competitors or attackers could flood the API, crashing the sales platform (Denial of Service) or scraping all data.

Remediation

Implement Rate Limiting (e.g., 100 req/min) and Throttling. Use API Gateways to enforce quotas per vendor API key.